Prompt: Build a Badminton Rotation & Points Web App

Create a clean, mobile-friendly web app for managing a casual but competitive **2v2 badminton rotation game**.

The app should automatically create teams, track individual points, track how many games each person has played, manage resting players, avoid repeated partners, and gradually make matches more competitive as more rounds are played.

The user should also be able to **manually force specific players to partner together for a particular round**, while the app automatically builds the remaining teams around those custom pairings.

1. Player Setup

Create a setup screen where I can enter any number of player names.

The order in which I enter the players represents their **estimated skill level**.

Example:

1. Seag
2. Phil
3. Remy
4. Aurel
5. Sam
6. Nadia
7. Kayla
8. Oscar

The first player entered is considered the strongest, and the last player entered is considered the weakest.

Call this value the player's:

Initial Skill Rank

This initial ranking should mainly be used to create balanced matches during the first few rounds, before enough real game data exists.

Allow me to:

- Add players
- Remove players
- Reorder players using drag-and-drop
- Start a new session
- Reset all session data

The app should work with different numbers of players.

Examples:

- 8 players + 2 courts = everyone plays
- 10 players + 2 courts = 8 play, 2 rest
- 13 players + 3 courts = 12 play, 1 rests
- 14 players + 3 courts = 12 play, 2 rest

Allow me to choose the **number of courts** before starting.

Each court always has 4 players.

Automatically calculate:

```
Players Playing = Courts × 4
```

```
Resting Players = Total Players − Players Playing
```

---

2. Scoring System

Every match has a **combined total score of 21**.

For example:

13–8
14–7
11–10
16–5

After the game, I should only need to enter:

- Winning team
- Winning score

The app automatically calculates:

```
Losing Score = 21 − Winning Score
```

Every player receives their team's score as individual points.

Example:

Seag + Phil beat Remy + Nadia 14–7.

Then:

Seag gets +14
Phil gets +14
Remy gets +7
Nadia gets +7

Individual scores accumulate throughout the session.

---

3. Live Leaderboard

After every round, automatically update a leaderboard showing:

| Rank | Player | Total Points | Games Played | Average Points/Game |

Sort primarily by total points.

Also calculate:

`Average Points Per Game = Total Points ÷ Games Played`

This is useful because some players may have rested more often or joined later.

Highlight:

- Top 3 players
- Players with fewer games than the current session average

---

4. Games Played Tracking

Track exactly how many games every player has played.

Playing-time fairness is extremely important.

The app should try to keep the difference between the highest and lowest number of games played as small as possible.

When choosing resting players, prioritize players who have played the **most games**.

Example:

If:

Seag = 6 games
Phil = 6
Remy = 5
Nadia = 5

Then Seag and Phil should generally rest before Remy or Nadia.

Whenever mathematically possible, bring everyone back to an equal number of games.

Display:

Games Played

and

Rest Count

for every player.

---

5. Resting Players

Clearly show the resting players every round.

Example:

RESTING

Seag
Phil

Keep a complete rest history:

Round 1 — Aurel
Round 2 — Rafi
Round 3 — Nadia
Round 4 — Seag, Phil

Avoid giving someone a second rest before other players have received their first rest unless required to keep games played balanced.

---

6. Partner History

Track every partnership.

Example:

Seag has partnered with:

- Nadia
- Aurel
- Phil

- Kayla

The system should strongly avoid repeated partners.

During early rounds, treat avoiding repeated partners as one of the highest priorities.

If repeat partners eventually become unavoidable, prefer:

1. The partnership that happened longest ago
2. The partnership with the fewest previous repeats
3. Never repeatedly assign the same partnership in nearby rounds if another reasonable option exists

Create a hidden or visible **partner matrix** showing how many times every pair has partnered.

---

7. Opponent History

Also track who has played against whom.

Opponent repeats are allowed more often than partner repeats, but try to create variety.

Priority should be:

Avoid repeated partners > avoid repeated opponents

Once competitive balancing becomes important, opponent repetition may be allowed if it creates a much better match.

---

8. Matchmaking Phases

The matchmaking algorithm should change throughout the session.

Rounds 1–4: Variety + Initial Skill Balance

Use the player's initial skill ranking heavily.

Goals:

1. Make reasonably balanced teams using the skill ranking entered at the start.
2. Avoid repeated partners.
3. Randomize enough that the session still feels social.
4. Vary opponents.
5. Keep games played/rests fair.
6. Do NOT aggressively use the points leaderboard yet.

A useful approach is to approximately pair stronger players with weaker players.

For example, with players ranked 1–8:

1 + 8
2 + 7
3 + 6
4 + 5

But do not make this exact arrangement every round.

Add controlled randomness while keeping approximate team strength balanced.

Rounds 5+: Competitive Mode

After approximately 4 rounds, there is enough real game data.

Start using cumulative performance more heavily.

Goals:

1. Keep playing time fair.
2. Avoid repeated partners.
3. Create balanced teams.
4. Make stronger-performing players increasingly play **against other stronger-performing players**.
5. Give lower-ranked players opportunities to climb by pairing them with stronger players.
6. Keep matches close and competitive.

A strong general method:

- Identify higher-performing and lower-performing players.
- Pair strong + weaker players where useful to balance team strength.
- Then match teams whose combined strength is similar.
- Prefer putting the highest-performing players on opposite sides of the net rather than always partnering them.

Example:

Current player strengths:

A = 80
B = 75
C = 70
D = 68
E = 60
F = 58
G = 50
H = 45

Possible teams:

A + H = 125
B + G = 125

C + F = 128
D + E = 128

Then:

Court 1: A + H vs B + G

Court 2: C + F vs D + E

This allows high players to compete against one another while weaker players still have a strong teammate.

---

9. Player Strength Calculation

Do not use cumulative points alone because players may have played different numbers of games.

Create an internal **Current Strength Score**.

Initially, Current Strength should heavily use the manually entered Initial Skill Rank.

As more games are played, gradually shift toward real performance.

Possible model:

Early rounds:

```
70% Initial Skill + 30% Game Performance
```

Middle rounds:

```
40% Initial Skill + 60% Game Performance
```

Later rounds:

```
20% Initial Skill + 80% Game Performance
```

Game Performance should consider:

- Average points per game
- Total points
- Recent results if useful

The exact formula can be improved if there is a better statistical approach.

The goal is not to create a professional ranking system. The goal is simply to produce **fun and balanced games**.

---

10. Custom / Forced Pairing

This is an important feature.

Before generating the next round, provide a section called:

Custom Pairings

I should be able to select two players and force them to play together for that round.

Example:

Force Pair Seag + Makayla

Then generate the rest of the teams automatically around that pairing.

Allow multiple forced pairs if possible.

Example:

Force:

- Seag + Makayla
- Calvin + Aurel

The matchmaking algorithm then creates the remaining teams while respecting:

- Forced partnerships
- Number of courts
- Rest fairness
- Balance
- Partner history
- Opponent variety

If a forced pairing conflicts with the normal algorithm, the forced pairing always takes priority.

Show a small warning if the manual pairing creates a heavily unbalanced match.

Example:

⚠️ This custom pairing may create an uneven matchup.

But still allow me to proceed.

Also allow me to manually force two teams to play against each other.

Example:

Seag + Aurel
VS
Remy + Titi

The remaining courts should then be generated automatically.

---

11. Manual Team Editing

After teams are generated, let me manually modify them before confirming the round.

Features:

- Drag players between teams
- Swap two players
- Change resting players
- Lock a team
- Lock an entire court matchup

Once I press:

Confirm Round

the teams become official and are added to partner/opponent history.

Do not update history while I am still editing the suggested round.

---

12. Round Screen

The active round screen should look something like:

Round 6

RESTING

Valen
Sam

Court 1

Makayla + Seag
VS
Calvin + Aurel

[Enter winning score]

Court 2

Hansen + Oscar
VS
Ong + Kayla

[Enter winning score]

Provide large buttons optimized for phone use.

After entering results for all courts, press:

Finish Round

Then automatically:

1. Calculate losing scores.
2. Add individual points.
3. Update games played.
4. Update leaderboard.
5. Update partner history.
6. Update opponent history.
7. Update rest history.
8. Generate suggested teams for the next round.

---

13. Result Input

Make result entry extremely simple.

For each court, show both teams as buttons.

Example:

Court 1

[Seag + Aurel]
VS
[Rem + Titi]

Tap the winning team.

Then enter:

Winning Score: 13

Automatically display:

13–8

Require winning score to be between **11 and 20** if using the 21-total system.

However, allow an admin override if needed.

---

14. Previous Rounds

Create a history page.

Example:

Round 1

Seag + Nadia — 14
Phil + Kayla — 7

Rem + Aurel — 11
Sam + Oscar — 10

Resting: Hansen, Calvin

Round 2

etc.

Allow previous rounds to be edited in case I accidentally enter a result incorrectly.

If an old result is edited, automatically recalculate:

- All points
- Rankings
- Average points
- Games played if necessary

Do not corrupt later round history.

---

15. Session Summary

At the end of the session, show:

Final Leaderboard

Player
Total Points
Games Played
Average Points/Game
Wins
Losses

Also show fun statistics:

- Highest total points
- Highest average points/game
- Most wins
- Closest game
- Biggest winning margin
- Most different partners
- Most common partnership

---

16. Wins and Losses

Although the main ranking is based on points, also track:

- Wins
- Losses
- Win percentage

Do not make wins the primary ranking system unless I change the setting.

---

17. Matchmaking Priority System

When automatically creating the next round, use approximately this priority:

Highest Priority

1. Respect custom/locked pairings
2. Keep number of games played fair
3. Avoid repeated partners

Medium Priority

1. Balance team strength
2. In later rounds, make top players face other top players
3. Avoid repeated opponents

Lower Priority

1. Randomness/social variety

Do not blindly follow a simple formula if it produces awkward teams.

Use an optimization/scoring approach where many possible team combinations are evaluated and the combination with the lowest penalty is selected.

For example, assign penalties for:

- Repeat partner: very high penalty
- Player has too many games compared with others: very high penalty

- Large team strength difference: medium/high penalty
- Repeat opponent: small/medium penalty
- Same players meeting repeatedly recently: additional penalty

Then select the combination with the lowest overall penalty.

---

18. Match Balance Score

Before confirming a suggested match, optionally display a simple estimated balance indicator.

Example:

Court 1
Seag + Kayla vs Phil + Nadia

Estimated Balance: 94%

Do not make this overly technical.

Possible labels:

🟢 Very Balanced
🟡 Fair
🔴 Uneven

---

19. Interface

Design the app primarily for mobile because it will be used courtside.

Style:

- Clean
- Modern
- Minimal
- Large touch targets
- Easy to read outdoors
- Fast to operate between games

Main navigation:

Game Leaderboard Players History Stats Settings

The active game screen is the most important.

Avoid unnecessary animations.

---

20. Important Behavior

Whenever displaying standings, always sort them correctly.

Always show:

- All players
- Total points
- Games played
- Who is resting this round

Never hide players simply because they are resting.

Do not accidentally award points to someone who rested.

Do not count a game as played until the round result is confirmed.

If a new player joins midway through the session:

- Add them with 0 points
- Give them 0 games played
- Try to give them more playing opportunities until their games played become closer to the others
- Do not artificially give them points
- Use their chosen Initial Skill Rank when balancing their first games

---

21. Settings

Allow configurable settings for:

- Number of courts
- Combined match score, default = **21**
- Number of early/random rounds before competitive mode, default = **4**
- Partner-repeat strictness
- Opponent-repeat strictness
- Match balance importance
- Rest fairness importance
- Whether Initial Skill Rank continues influencing later rounds
- Ranking method:
- Total Points
- Average Points/Game
- Wins
- Custom

Default should remain:

Total Points

Main Goal

The app should feel like an intelligent badminton session manager.

I should only need to:

1. Enter players in approximate skill order.
2. Choose number of courts.
3. Start the session.
4. Accept or customize the suggested teams.
5. Enter the winning team and winning score after each match.

The app handles everything else automatically:

scoring + leaderboard + games played + rests + partner rotation + opponent variety + team balancing + competitive matchmaking.

The most important philosophy is:

Early session = social variety and fair mixing.

Later session = competitive, balanced games based on actual performance.

But throughout the entire session:

keep playing time fair and avoid repeated partners as much as possible.